ĐẠI HỌC BÁCH KHOA HÀ NỘI

TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

BÀI TẬP LỚN

Môn: IT3160 - Nhập môn Trí tuệ Nhân tạo

Đề tài: Bài toán tìm đường đi trên bản đồ

Nhóm: 20

Mã lớp học: 162278

Giáo viên hướng dẫn: PGS.TS Trần Đình Khang

Danh sách sinh viên thực hiện:

STT	Họ và tên	Mã sinh viên	Email
1	Nguyễn Đức Nhẫn	20235182	nhan.nd235182@sis.hust.edu.vn
2	Đoàn Xuân Công Đạt	20235030	dat.dxc235030@sis.hust.edu.vn
3	Nguyễn Tùng Dương	20230023	duong.nt230023@sis.hust.edu.vn
4	Nguyễn Mai Huy	20235107	huy.nm235107@sis.hust.edu.vn
5	Hoàng Kim Tiến	202416367	tien.hk2416367@sis.hust.edu.vn

Hà Nội, ngày 14 tháng 12 năm 2025

Mục lục

1 Mô tả bài toán **3**

2 Biểu diễn bài toán **3**

2.1 Trạng thái bài toán 3

2.2 Không gian trạng thái 4

2.3 Trọng số cạnh 5

2.4 Phát biểu bài toán 6

3 Phương pháp tìm kiếm **7**

4 Công cụ thực hiện **11**

5 Kết quả **12**

Danh sách hình vẽ

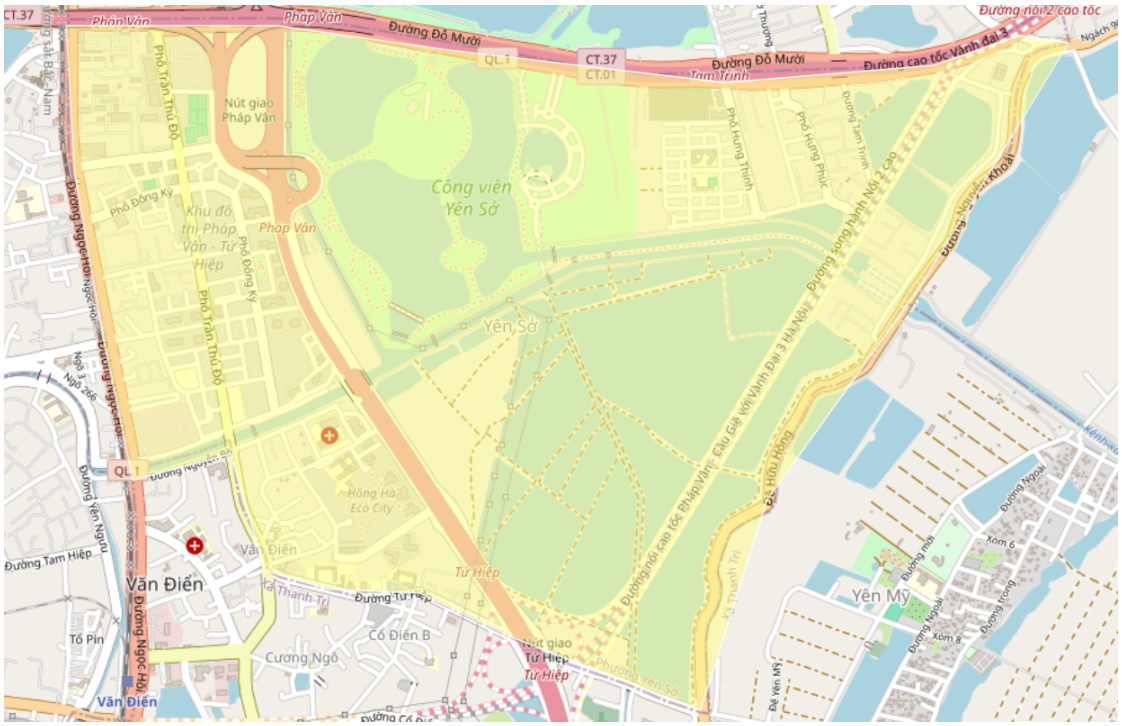
1 Mô phỏng khu vực bản đồ Yên Sở 3
2 Mô phỏng các trạng thái (giao lộ) trên bản đồ 4
3 Hình ảnh đường đi trước khi cải tiến thuật toán. 9
4 Hình ảnh đường đi sau khi cải tiến thuật toán. 10
5 Hình ảnh tuyến đường tìm đường khi chưa cấm đường. 12
6 Hình ảnh tuyến đường tìm được sau khi cấm đường. 13
7 Hình ảnh tuyến đường tìm được. 14

1 Mô tả bài toán

Xét một khu vực bản đồ thuộc phường Yên Sở, thành phố Hà Nội. Bài toán yêu cầu tìm đường đi giữa hai điểm A và B bất kỳ, đồng thời hiển thị đường đi đó trên bản đồ.

Ràng buộc:

- Chỉ được phép di chuyển trên các tuyến đường, không được phép đi xuyên qua các công trình.
- Không được phép đi trên cùng một tuyến đường nhiều hơn một lần.
- Có thể xem xét loại bỏ 1 tuyến đường để chọn một tuyến dường khác thỏa mãn do điều kiện khách quan (thời tiết, tắc đường, đường một chiều...)



Hình 1: Mô phỏng khu vực bản đồ Yên Sở

2 Biểu diễn bài toán

2.1 Trạng thái bài toán

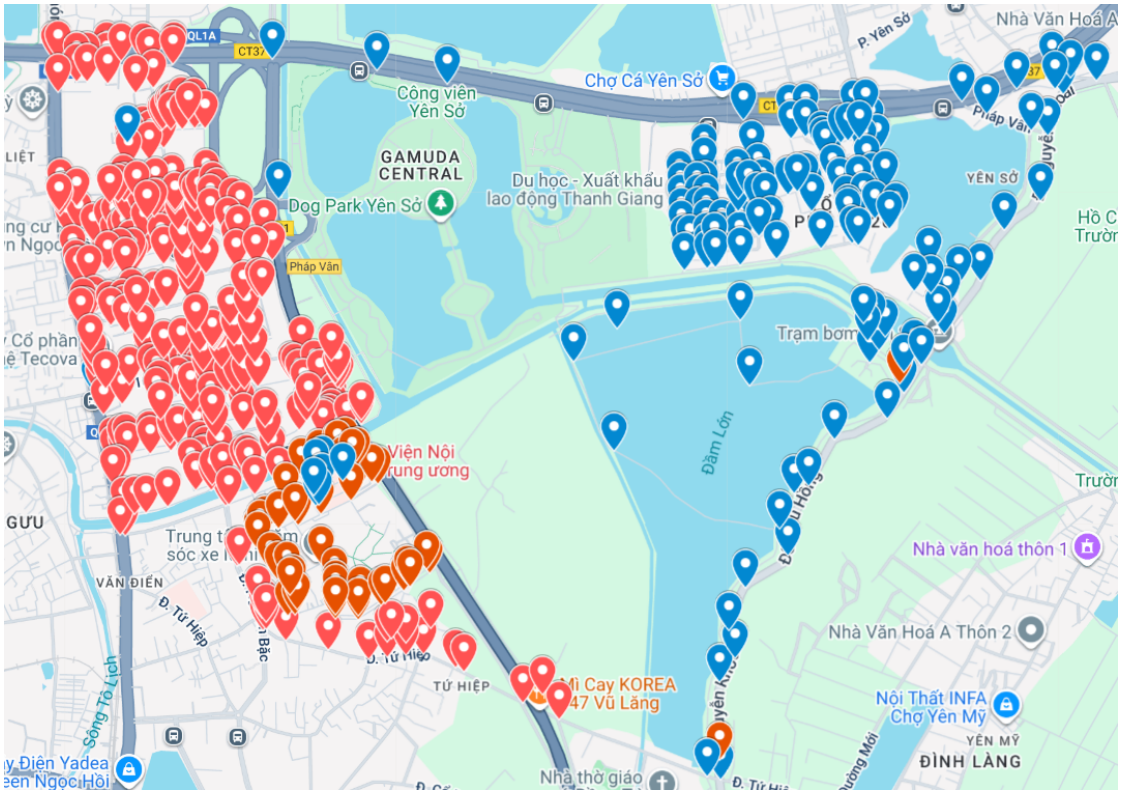
Trạng thái được biểu diễn dưới dạng tập hợp các cặp tọa độ:

$$S = \{(lat, lng)\}$$

Trong đó:

- *lat*: Vĩ độ
- *lng*: Kinh độ

Mỗi phần tử trong tập S biểu diễn một **giao lộ** trên bản đồ. Hình 2 minh họa các giao lộ (các chấm tròn).



Hình 2: Mô phỏng các trạng thái (giao lộ) trên bản đồ

2.2 Không gian trạng thái

Bản đồ được mô hình hóa dưới dạng **mặt phẳng** R^2 với hệ Descarte (x, y), trong đó:

- Tập hợp các vật thể không thể đi xuyên qua, các công trình được biểu diễn dưới dạng đa giác kín P_i.

- **Không gian khả dụng** được xác định bởi:

$$F = \mathbb{R}^2 \setminus \bigcup_{i=1}^{k} P_i$$

Đường đi π được xem là hợp lệ nếu và chỉ nếu nó là một đường liên tục thỏa mãn:

$$\pi \subseteq F \cup \{s, t\}$$

với s và t lần lượt là điểm bắt đầu và điểm kết thúc.

Tập các vị trí hợp lệ (các điểm có thể đi qua) được định nghĩa là:

$$\mathcal{N} = \{(x, y) \in F \mid (x, y) \text{ là điểm khả dụng trên bản đồ}\}$$

Từ tập $\mathcal{N}$, ta định nghĩa đồ thị có hướng:

$$G = (V, E)$$

trong đó:

- $V = \{v_1, v_2, \ldots, v_n\}$: Tập hợp các đỉnh (giao lộ). Từ Hình 2, Tổng cộng có 476 giao lộ, tức là $|V| = 476$.
- $E = \{(v_i, v_j) \mid v_i, v_j \in V\}$: Tập hợp các cạnh (tuyến đường nối hai giao lộ)

Từ một giao lộ v_i, chỉ có thể di chuyển đến các đỉnh v_j **kề** với nó sao cho đường nối $v_i \to v_j$ là hợp lệ (không bị cấm) và không có điểm trung gian. Điều này đảm bảo điều kiện "không đi xuyên qua công trình" và chỉ di chuyển dọc theo các tuyến hợp lệ.

2.3 Trọng số cạnh

Mỗi cạnh $(v_i, v_j) \in E$ được gán một trọng số $w(v_i, v_j) > 0$ biểu thị **độ dài hoặc chi phí di chuyển** giữa hai đỉnh. Ngoài ra, mỗi cạnh còn có một biến logic (bool):

$$F(v_i, v_j) \in \{0, 1\}$$

với ý nghĩa:

- $F = 1$: Tuyến đường hợp lệ, được phép đi qua
- $F = 0$: Tuyến đường bị cấm, không thể đi được

Như vậy, tập cạnh hợp lệ được định nghĩa:

$$E' = \{(v_i, v_j) \in E \mid F(v_i, v_j) = 1\}$$

2.4 Phát biểu bài toán

- **Đầu vào:** Đồ thị có hướng $G = (V, E)$ cùng trọng số $w(v_i, v_j)$. Tập $M = \{v_{i_1}, v_{i_2}, \ldots, v_{i_m}\}$ là các điểm bắt buộc phải đi qua. Hai đỉnh đặc biệt $s, d \in V$ lần lượt là **điểm xuất phát** và **điểm đích**.

- **Đầu ra:** Tìm một đường đi $P = (v_{p_1}, v_{p_2}, \ldots, v_{p_k})$ sao cho:

$$v_{p_1} = s, \quad v_{p_k} = d, \quad M \subseteq P$$

  và P là một chuỗi cạnh hợp lệ trong E'.

- **Ràng buộc:**

$$v_{p_i} \in V, \quad (v_{p_i}, v_{p_{i+1}}) \in E', \quad \forall i \in [1, k-1]$$

- **Mục tiêu:** Tối thiểu hóa tổng độ dài đường đi:

$$\min_P \sum_{i=1}^{k-1} w(v_{p_i}, v_{p_{i+1}})$$

Mục tiêu phụ là tối thiểu hóa thời gian tìm kiếm bằng cách sử dụng thuật toán tìm kiếm hiệu quả như A*, UCS hoặc Dijkstra.

3 Phương pháp tìm kiếm

Đầu vào bài toán: Người dùng click vào 2 điểm trên bản đồ

Đầu ra bài toán: Đường đi giữa 2 điểm không đi qua các đường bị cấm (chọn các đường từ controller)

Phương pháp tìm kiếm:

Bước 1: Xác định trạng thái đầu, trạng thái đích

- Xác định trạng thái đầu:
  - Kiểm tra tất cả các cặp giao lộ kề nhau, chọn các cặp giao lộ mà điểm xuất phát nằm trên đoạn đường giới hạn bởi cặp giao lộ này.
  - Kiểm tra tất cả các cặp giao lộ kề nhau, chọn các cặp giao lộ mà điểm xuất phát nằm trên đoạn đường giới hạn bởi cặp giao lộ này.
  - Nếu không tìm được cặp giao lộ thỏa mãn yêu cầu trên thì trạng thái đầu sẽ là giao lộ gần nhất đối với điểm xuất phát
- Xác định trạng thái đích:
  - Kiểm tra tất cả các cặp giao lộ kề nhau, chọn các cặp giao lộ mà điểm kết thúc nằm trên đoạn đường giới hạn bởi cặp giao lộ này.
  - Trạng thái đích sẽ là giao lộ gần nhất với điểm kết thúc trong các cặp giao lộ tìm được..
  - Nếu không tìm được cặp giao lộ thỏa mãn yêu cầu trên thì trạng thái đích sẽ là giao lộ gần nhất đối với điểm kết thúc.

Bước 2: Tìm đường đi từ trạng thái đầu đến trạng thái đích

- Sử dụng thuật toán **A*** với hàm đánh giá:

$$f(n) = g(n) + h(n)$$

Trong đó:

  - g(n) là tổng khoảng cách đường đi thực tế từ trạng thái đầu đến trạng thái đang xét

– h(n) là khoảng cách ước lượng (khoảng cách đường chim bay) từ trạng thái đang xét đến trạng thái đích, được tính theo công thức Haversine:

Giả sử trạng thái đích là **(lat2, lng2)**, trạng thái hiện tại là **(lat1, lng1)**, ta có:

$$h(n) = 2 * 6731000 * \arcsin\sqrt{A + B}$$

Trong đó:

$$A = \sin^2(\frac{\pi}{180} * \frac{lat_2 - lat_1}{2})$$

$$B = \cos(\frac{\pi}{180} * lat_1) * \cos(\frac{\pi}{180} * lat_2) * \sin^2(\frac{\pi}{180} * \frac{lng_2 - lng_1}{2})$$

Mô tả chi tiết thuật toán:

* Lưu trữ và cập nhật liên tục 2 tập hợp *Open* và *Close*, trong đó *Open* lưu các trạng thái sẽ được xét, còn *Close* lưu các trạng thái đã được xét đến rồi và sẽ không được xét lại. Tập *Open* được khởi tạo với trạng thái đầu n_0, tập *Close* được khởi tạo rỗng.
* Lưu trữ 1 mảng $d[n]$ là tổng khoảng cách đường đi thực tế từ trạng thái đầu đến trạng thái n, 1 mảng $trace[n]$ là trạng thái ngay liền trước trạng thái n trong đường đi cuối cùng tìm được. Khởi tạo $d[n0] = 0$, còn lại thì bằng inf.
* Định nghĩa 1 hàm $distance(n, n*)$ là khoảng cách giữa 2 trạng thái n và n^*.
* Khi tập *Open* vẫn còn chứa ít nhất 1 trạng thái, thực hiện liên tục các công việc sau:
  · Chọn trạng thái n có hàm đánh giá $f(n)$ nhỏ nhất trong tập *Open*.
  · Xét các hành động từ trạng thái n sang 1 trạng thái n^*, nếu n^* không thuộc *Close* thì đưa n^* vào *Open*.
  · Cập nhật $d[n^*] = min(d[n^*], d[n] + distance(n, n^*))$ và $trace[n^*] = n$ nếu $d[n^*]$ có sự thay đổi.
  · Đưa n ra khỏi *Open* và đưa n vào *Close*.
  · Nếu tập *Open* hoặc *Close* đã chứa trạng thái đích thì kết thúc tìm kiếm

* Dựa vào mảng $trace[]$ đã được tính toán, ta có thể tìm được 1 đường đi từ trạng thái đầu đến trạng thái đích. Thêm điểm xuất phát và điểm kết thúc do người dùng chọn vào đầu và cuối đường đi tìm được, ta đã có được 1 đường đi hoàn chỉnh giữa 2 điểm và đảm bảo việc không đi xuyên qua bất cứ công trình nào

Bước 3: Cải tiến thuật toán

- Tình huống: Giả sử A, B lần lượt là 2 điểm được người dùng lựa chọn. Gọi n_0 là trạng thái đầu, n_1 là trạng thái tiếp theo,.... Đường đi tìm được sẽ có dạng:

$$A \to n_0 \to n_1 \to ... \to n_{k-1} \to n_k \to B$$

  Nếu A nằm trên đoạn đường nối giữa n_0 và n_1, thì đường đi tìm được sẽ vi phạm ràng buộc: **"Không được phép đi trên cùng một tuyến đường nhiều hơn 1 lần"**.

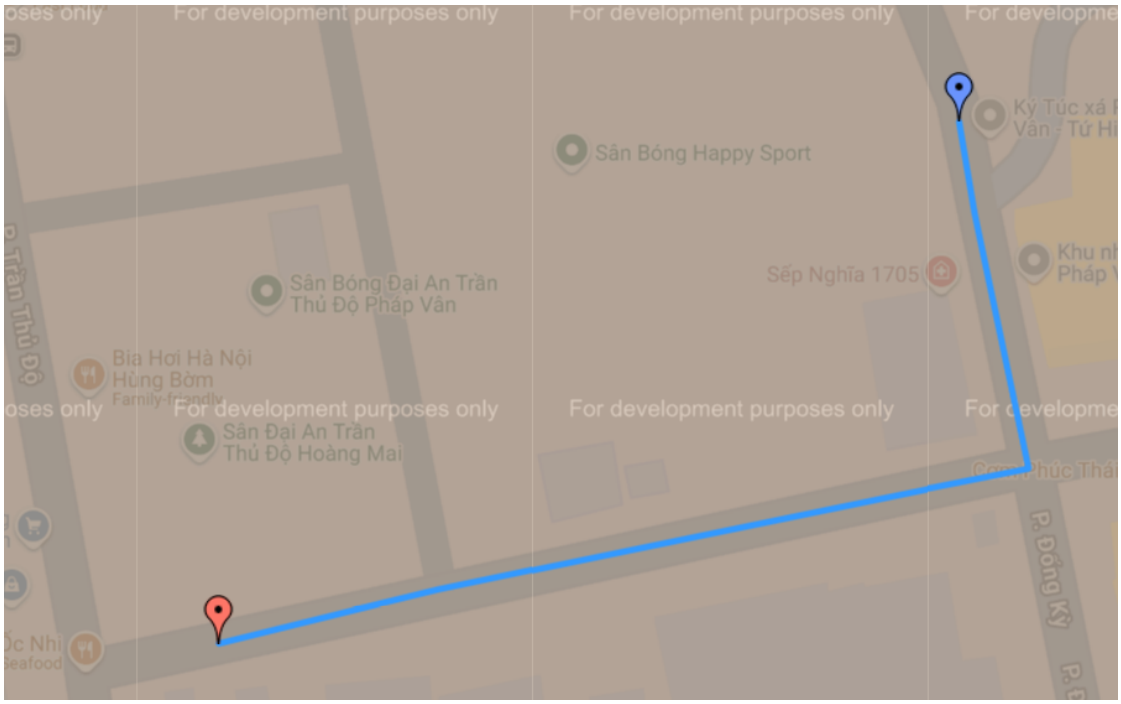
- Hình ảnh minh họa:

Hình 3: Hình ảnh đường đi trước khi cải tiến thuật toán.

- Giải pháp:

- Kiểm tra xem A có nằm giữa n_0 và n_1 không, bằng cách so sánh tổng khoảng cách từ A đến n_0 và khoảng cách từ A đến n_1, với khoảng cách từ n_0 và n_1. Nếu 2 giá trị này xấp xỉ bằng nhau thì A nằm giữa n_0 và n_1 và ta cần phải loại bỏ n_0 khỏi đường đi.
- Tương tự, ta cũng cần thực hiện cải tiến trên với các trạng thái cuối: n_{k-1}, n_k và điểm kết thúc B. Nếu B nằm giữa n_{k-1} và n_k thì ta cần loại bỏ n_k khỏi đường đi.

- Hình ảnh đường đi tìm được sau khi cải tiến thuật toán: Như vậy, sau khi thực



Hình 4: Hình ảnh đường đi sau khi cải tiến thuật toán.

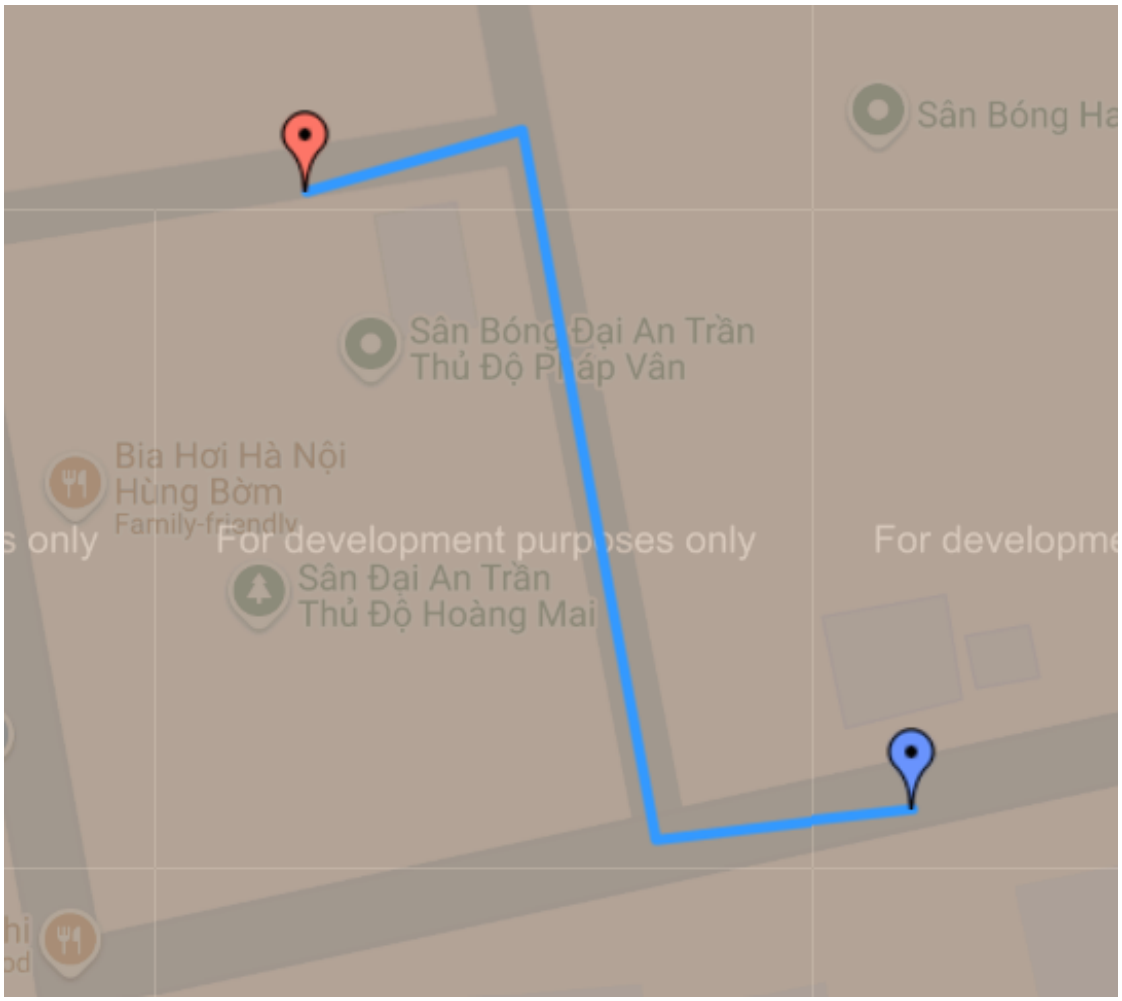
hiện cải tiến, đường đi tìm được giữa A và B đã chắc chắn thỏa mãn tất cả các rang buộc. Lưu ý rằng ta không cần kiểm tra đường đi giữa các trạng thái trung gian từ n_1 đến n_{k-1} nhờ vào tính tối ưu của thuật toán A^*.

4 Công cụ thực hiện

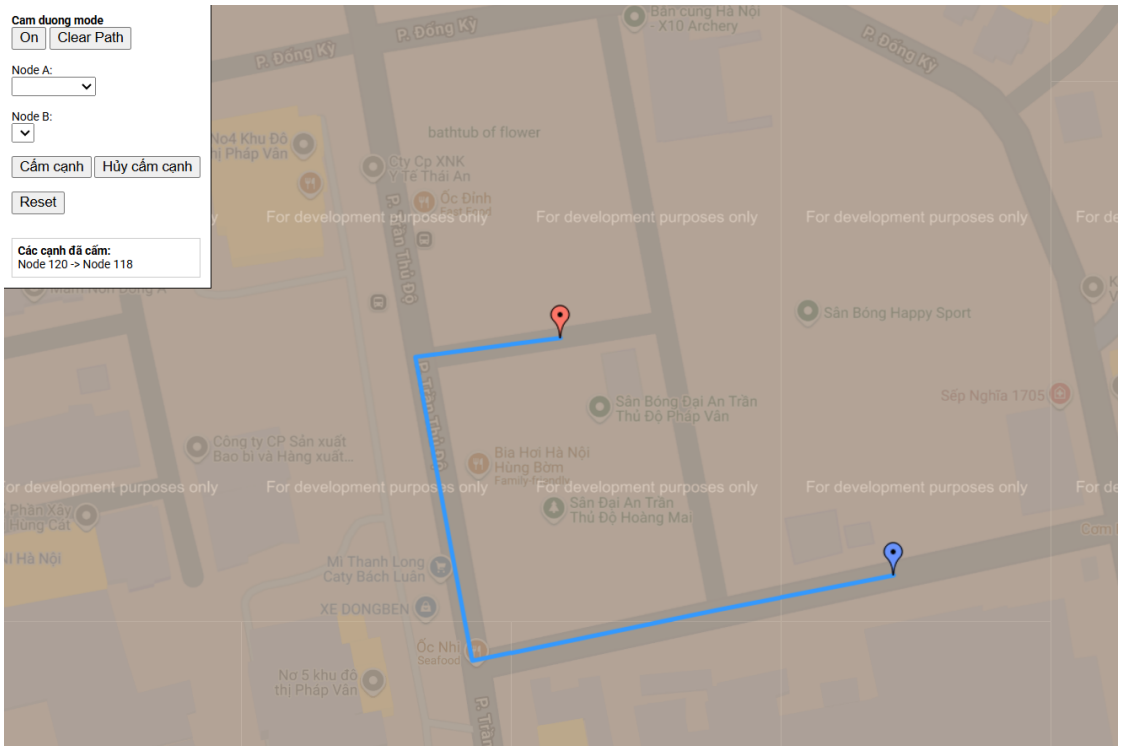
- Ngôn ngữ lập trình: JavaScript. Đây là ngôn ngữ được hỗ trợ bởi API của Google Map: Maps JavaScript API.
- Maps JavaScript API hỗ trợ hệ thống trong việc tạo một bản đồ có thể tương tác với người dùng, cụ thể các công việc mà API này hỗ trợ:
  - Tạo một vùng giới hạn mảnh bản đồ của phường Hàng Bồ, dựa trên các tọa độ được cung cấp. Vùng giới hạn này được tô màu để người dùng nhận biết được những vị trí có thể tương tác được.
  - Tạo 1 marker với mỗi vị trí mà người sử dụng click lên bản đồ. Vị trí xuất phát được đánh dấu màu đỏ, vị trí kết thúc được đánh dấu màu xanh.
  - Mô phỏng đường đi trên bản đồ bằng cách tạo đường thẳng nối các điểm trên đường đi với nhau. Khi người dùng di chuột vào đường đi, có thể hiển thị độ dài của đường đi đó (độ dài đường đi được tính toán trước).

5 Kết quả

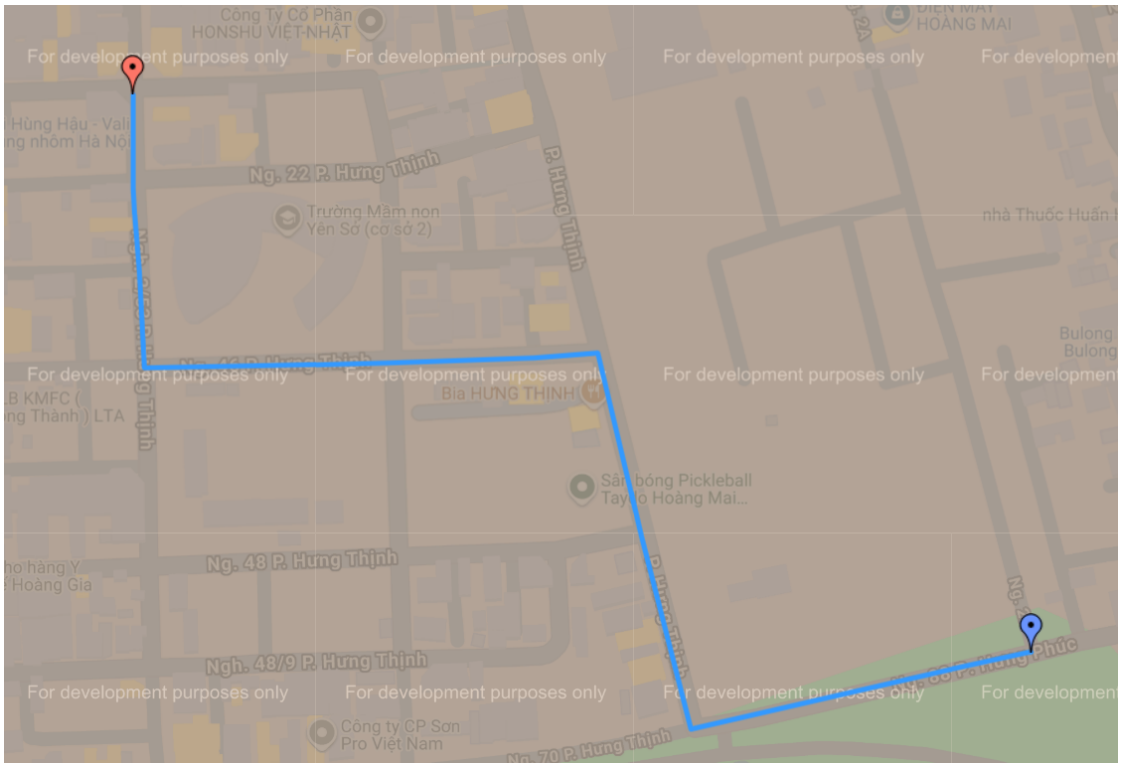
Hình ảnh minh họa kết quả bài toán



Hình 5: Hình ảnh tuyến đường tìm đường khi chưa cấm đường.



Hình 6: Hình ảnh tuyến đường tìm được sau khi cấm đường.



Hình 7: Hình ảnh tuyến đường tìm được.

Địa chỉ trang web

Github